

Vector calculus in DL

You are my tutor for vector calculus for deep learning, covering differentiation of vectors and matrices, Jacobians, gradients, Hessians, chain rule for deep nets, and practical backpropagation.

I also want hands-on NumPy coding exercises to help build intuition and skill.

This should be an interactive tutoring session. In every reply:

- Keep responses focused and reasonably short (3–6 paragraphs).
- Adapt dynamically to my understanding: if I misunderstand, briefly clarify and ask a grounding question; if I answer well, increase difficulty.
- Always end with a specific question or a small task I must answer before you continue.
- Include at least one short NumPy coding exercise every 2–3 messages.
- Do not move on to the next topic until I say so.

Overall Story I Want to Learn

Teach vector calculus as used in modern deep learning workflows, following this outline unless I say otherwise:

1. Big-picture motivation
 - Why DL requires vector calculus: gradients, Jacobians, chain rule, backprop.
 - Scalar $\rightarrow$ vector $\rightarrow$ matrix differentiation as a natural progression.
2. Core objects
 - Gradients, Jacobians, Hessians.
 - Shapes and dimensions intuition (e.g., gradient is same shape as input).
 - Inner products and directional derivatives.
3. Differentiation rules in vector form

- Chain rule for vector functions.
- Product rules for matrix–vector expressions.
- Common derivatives used in neural nets (ReLU, softmax, linear layers, norms).
- 4. Backpropagation through computations
 - Computational graph view.
 - Local Jacobians and how they compose.
 - Examples from linear layers, activations, softmax cross-entropy.
- 5. Practical NumPy differentiation
 - Implementing gradients from scratch for simple functions.
 - Numerical gradient checking.
 - Mini backprop through a 2-layer MLP.
- 6. Connecting to deep learning practice
 - Why PyTorch/JAX autodiff works.
 - Relationship between Jacobian–vector products and reverse-mode AD.
 - Efficiency considerations.

Step 1: First Response Instructions

Your first response should include:

1. A two-paragraph big-picture introduction, explaining:
 - Why DL optimization is impossible without vector calculus.
 - How gradients in high dimensions behave and why shapes/Jacobians matter.
 - Why backprop is essentially repeated applications of the vector chain rule.
2. A short roadmap (5–7 bullets) drawn from the “Overall Story” above.
3. End with one short concept-check question, such as:

“What is the difference between a gradient and a Jacobian?”

Wait for my answer before continuing.

Step 2: Teaching Each Topic

For each topic in the roadmap, use a four-layer structure:

1. Intuition

Explain the idea in clear language: geometric meaning, shapes, why it's needed for DL (optimization, backprop, stability).

2. Key Formal Setup

Introduce minimal notation:

- Vector functions $f: \mathbb{R}^n \rightarrow \mathbb{R}$ or $g: \mathbb{R}^n \rightarrow \mathbb{R}^m$.
- Jacobian shapes.
- Example derivatives (e.g., Ax , norms, nonlinearities).

Keep formulas short and tied to intuition.

3. Tiny Example

Provide a tiny numeric or symbolic example, plus a NumPy mini-exercise every 2–3 messages. Exercises should be short (5–10 lines max), such as:

- Compute numerical gradients by finite differences.
- Verify the gradient of $f(x) = x^T A x$.
- Backprop through a 1-layer network.
- Implement Jacobian-vector products manually.

4. Understanding Check

Ask me 1–2 questions: one conceptual, optionally one computational.

Then stop and wait for my answer.

Only continue when I confirm I'm ready for the next topic.

Step 3: Final Wrap-Up

When I say I'm ready for review, give a compact summary covering:

- How vector calculus underlies gradients, Jacobians, backprop.
- How NumPy exercises connect to the math.
- Why modern autodiff frameworks are efficient implementations of the same principles.
- A final list of "the 10 most important derivative patterns in deep learning."

Begin Now

Start with Step 1:

Provide the two-paragraph big-picture introduction, the roadmap, and a final concept-check question.

Deep learning is basically “optimize a giant function with millions of inputs.” Those inputs are the parameters (weights) of your network, and the function is the loss. To make the loss small, you need to know **which direction in parameter space decreases it fastest**—that’s exactly what gradients and their higher-dimensional cousins (Jacobians, Hessians) tell you. Without vector calculus, you’d be stuck randomly poking weights and hoping things improve. With it, you can run gradient descent and its fancy variants (Adam, RMSProp, etc.) in very high dimensions, and be confident you’re moving in locally good directions.

But in deep learning we never deal with just single numbers; almost everything is **vectors and matrices**: inputs, hidden activations, parameters, batched data. That’s why shapes matter so much. The Jacobian keeps track of how each output coordinate depends on each input coordinate. Backpropagation is, under the hood, **repeated application of the vector chain rule** along a computational graph: local derivatives (small Jacobians) get multiplied and composed as you flow gradients from the loss back to earlier layers. If you understand the shapes and the chain rule in vector form, backprop stops feeling like magic and starts feeling like book-keeping.

Here’s the rough roadmap we’ll follow (and we won’t move on until you say you’re ready):

- Why optimization in DL is all about gradients, Jacobians, and the chain rule.
- Core objects: gradients, Jacobians, Hessians, and their shapes.
- Differentiation rules for common DL building blocks (linear layers, ReLU, softmax, norms).
- Backpropagation through computational graphs using local Jacobians.
- Practical NumPy work: manual gradients, numerical checks, and a tiny 2-layer MLP backprop.
- How this all connects to PyTorch/JAX autodiff and why they’re efficient.

Before we dive deeper: at a high level, **what is the difference between a gradient and a Jacobian?**

gradient is vector, jacobian is matrix

Great — that’s the essential distinction. Let me sharpen it just a bit so we’re fully aligned before moving on:

- The **gradient** applies when your function outputs a *scalar*:

$$f : \mathbb{R}^n \rightarrow \mathbb{R}.$$

Its derivative is a **vector** in $\mathbb{R}^n$, telling you how sensitive the scalar is to each input coordinate.

- The **Jacobian** applies when your function outputs a *vector*:

$$g : \mathbb{R}^n \rightarrow \mathbb{R}^m.$$

Its derivative is an $m \times n$ **matrix**, where each row is the gradient of one output component.

This distinction becomes crucial in deep learning because backprop flows *scalar loss gradients* backward through layers whose local derivatives are often *Jacobian matrices*. Backprop’s efficiency comes from never forming full Jacobians explicitly — instead it uses Jacobian–vector products, which is why shape intuition matters so much.

Quick check before we advance to Topic 2 (Core Objects)

Imagine a function $g(x) = Ax$, where A is a fixed $m \times n$ matrix.

Question:

If $g : \mathbb{R}^n \rightarrow \mathbb{R}^m$, what is its Jacobian $J_g(x)$?

(Just describe its shape and what it equals — no need for computation yet.)

A.T

Close — and this is a *super* common mix-up — so let’s slow down and ground the idea.

For a vector function

$$g(x) = Ax, \quad A \in \mathbb{R}^{m \times n},$$

each output component is

$$g_i(x) = \sum_{j=1}^n A_{ij} x_j.$$

So the derivative of g_i w.r.t. x_j is just A_{ij} .

That means the **Jacobian is literally the matrix A** itself.

- **Shape:** $m \times n$
- **Value:** $J_g(x) = A$
- Not transposed.

Your answer A^T would correspond to a different derivative convention (sometimes used in physics), but in ML/backprop we always use the convention:

$$[J_g]_{ij} = \frac{\partial g_i}{\partial x_j}.$$

Tiny grounding question

Why *should* the Jacobian of Ax be A , not A^T ?

(Think about which output index depends on which input index.)

Keep it in one or two sentences.

because the Jacobian of Ax at $[i][j]$ is the derivative of $Ax[i] = \sum_j A_{ij} * x_j$ wrt $x[j]$ which is just A_{ij}

Exactly — that's the right reasoning.

You matched indices correctly: output g_i depends on input x_j through the coefficient A_{ij} , so the Jacobian must store those in the same layout → **the Jacobian is A** .

Great. Let's move on to **Topic 2: Core Objects (gradients, Jacobians, Hessians)**.

2.1 Intuition: What these objects *mean*

- The **gradient** of a scalar function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ tells you the *direction of steepest ascent*. It's the vector that changes the scalar output the most. This is why gradient descent uses the negative gradient.
- The **Jacobian** of a vector function $g : \mathbb{R}^n \rightarrow \mathbb{R}^m$ tells you how each output component changes with each input component. In deep nets, every layer computes a vector output, so its derivative is conceptually a Jacobian.
- The **Hessian** of a scalar function f is the Jacobian of the gradient: an $n \times n$ matrix of second derivatives. It measures curvature, which matters for optimization stability (Newton methods, saddle-point escape, etc.).

Geometrically:

- Gradient = slope vector.
- Jacobian = linear map approximating the entire function near a point.
- Hessian = curvature map.

2.2 Key Formal Setup

Let

$$f : \mathbb{R}^n \rightarrow \mathbb{R}, \quad g : \mathbb{R}^n \rightarrow \mathbb{R}^m.$$

- **Gradient:**

$$\nabla f(x) \in \mathbb{R}^n.$$

- **Jacobian:**

$$J_g(x) = \left[\frac{\partial g_i}{\partial x_j} \right]_{i,j} \in \mathbb{R}^{m \times n}.$$

- **Hessian:**

$$H_f(x) = \left[\frac{\partial^2 f}{\partial x_i \partial x_j} \right]_{i,j} \in \mathbb{R}^{n \times n}.$$

In backprop, we usually never build a full J_g ; we multiply the upstream gradient (a row vector) by the local Jacobian — that's the vector chain rule.

2.3 Tiny Example

Let

$$f(x) = \|x\|^2 = x^\top x.$$

- Gradient should be $\nabla f(x) = 2x$.
- Hessian should be $H_f(x) = 2I$.

Mini NumPy exercise (your first one):

Using NumPy, compute both:

1. The analytical gradient $2x$.
2. A numerical finite-difference gradient to verify it.

Use a small vector like `x = np.array([1.0, -2.0, 0.5])`.

(Just write the code; no need to run it.)

2.4 Understanding Check

Two questions:

1. Conceptual:

Why is the Hessian of $f(x) = \|x\|^2$ the same everywhere, independent of x ?

2. Coding:

Post your NumPy snippet for the finite-difference gradient check.

Answer both, and then we'll proceed.

the gradient of $f(x)$ is $2x$ so the second derivative is just a constant.

```
x = np.array([1.0, -2.0, 0.5])
y = x.T @ x
grad = 2*x
num_grad = np.gradient(y, x)
```

Nice on the conceptual part — exactly right.

$f(x) = x^\top x$ is a quadratic with constant curvature, so its second derivative does not depend on where you are in space. That's why the Hessian is $2I$ everywhere.

For the NumPy part, though, there's one important correction:

⚠ `np.gradient` **does *not* compute numerical gradients of scalar functions.**

`np.gradient` expects an array whose *values vary over a grid*, not a scalar function evaluated at one point. So:

python

```
np.gradient(y, x)
```

cannot work here — `y` is a scalar.

To compute a finite-difference gradient, you must perturb each coordinate of `x` separately and reevaluate the function. Something like:

python

```
eps = 1e-5
num_grad = np.zeros_like(x)
for i in range(len(x)):
    x_forward = x.copy(); x_forward[i] += eps
    x_backward = x.copy(); x_backward[i] -= eps
    f_forward = f(x_forward)
    f_backward = f(x_backward)
    num_grad[i] = (f_forward - f_backward) / (2 * eps)
```

This matches the definition:

$$\frac{\partial f}{\partial x_i} \approx \frac{f(x + \epsilon e_i) - f(x - \epsilon e_i)}{2\epsilon}.$$

Quick Check Before Moving to Differentiation Rules

One conceptual question:

Why do we re-evaluate the entire function $f(x)$ twice for each coordinate when computing numerical gradients?

(Answer in 1–2 sentences.)

you need to perturb both up and down

Exactly — and the reason we perturb *up and down* is to approximate the derivative symmetrically:

$$\frac{\partial f}{\partial x_i} \approx \frac{f(x + \varepsilon e_i) - f(x - \varepsilon e_i)}{2\varepsilon},$$

which cancels first-order error terms and yields a much more stable numerical estimate. Great — you're ready to move on.

Topic 3: Differentiation Rules in Vector Form

We'll now build the *toolkit* that deep learning uses constantly: vector chain rule, product rules, and common derivatives (ReLU, softmax, norms, linear layers). This directly sets up everything for backprop later.

3.1 Intuition

In deep nets, every layer is a vector function:

$$h^{(k)} = g^{(k)}(h^{(k-1)}).$$

During backprop, we repeatedly apply the **vector chain rule**, propagating a gradient from the scalar loss L backward through these transformations.

So the key intuition is:

- Each layer has a **local Jacobian** describing how its outputs change with respect to its inputs.

- Backprop multiplies these Jacobians (via Jacobian-vector products) to move gradients backward.
- This means knowing **shapes** and **derivative patterns** is more important than memorizing formulas.

Once you know the derivative of a linear layer, a ReLU, a softmax, and a norm, you basically know 90% of backprop.

3.2 Key Formal Setup: Vector Chain Rule

If

$$f : \mathbb{R}^m \rightarrow \mathbb{R}, \quad g : \mathbb{R}^n \rightarrow \mathbb{R}^m,$$

then

$$\nabla_x f(g(x)) = J_g(x)^\top \nabla_y f(y)|_{y=g(x)}.$$

Interpretation:

- The upstream gradient is a vector in $\mathbb{R}^m$.
- The local Jacobian J_g is an $m \times n$ matrix.
- Their product gives a gradient in $\mathbb{R}^n$.

This is exactly what backprop does layer by layer.

3.3 Tiny Example: Linear Layer

Let

$$g(x) = Ax + b.$$

Then:

- $J_g(x) = A$.
- Using the chain rule for $L(g(x))$:

$$\nabla_x L = A^\top \nabla_g L.$$

This $A^\top$ is why the gradient wrt a linear layer's input uses the **transpose**, even though the Jacobian itself is just A . This is a common point of confusion.

Mini NumPy Exercise

Write a short NumPy snippet that:

1. Samples a random $A \in \mathbb{R}^{3 \times 2}$, a random vector $x \in \mathbb{R}^2$.
2. Defines a scalar function

$$L = \|Ax\|^2.$$

3. Computes:
 - analytical gradient:

$$\nabla_x L = 2A^\top(Ax)$$

- numerical gradient via finite differences.

Only the code — no need to run it.

3.4 Understanding Check

Answer both:

1. Conceptual:

Why does the chain rule produce an $A^\top$ instead of A when backpropagating through $Ax + b$?

2. Computational:

Post your NumPy snippet for the gradient check.

When you're done, we'll move to nonlinear derivatives (ReLU, softmax, norms).

1. because the chain rule is taking the derivative with respect to the final loss which is a scalar.
2. `a = np.random.randn(2,3)` `x = np.random.randn(2)` `l = (A@x).T @ (A@x)` `grad = 2 * A.T @ (A@x)`. lets skip the numerical gradient via finite differences from now on

Good instincts on both points — but let me correct one subtle but important thing before we continue.

✓ 1. Conceptual answer

You're right that the chain rule involves the scalar loss, but the *real* reason the transpose appears is more specific:

Backprop multiplies an upstream gradient vector (shape m) by the Jacobian (shape $m \times n$).

$$\nabla_x L = J_g(x)^\top \nabla_g L.$$

For $g(x) = Ax$, the Jacobian is A , so:

$$\nabla_x L = A^\top \nabla_g L.$$

We transpose because the Jacobian enters the chain rule as $J^\top$ for scalar-output functions.

So your intuition is on the right track — backprop through a scalar loss uses *adjoints* / *transpose Jacobians* — but it's good to remember that the rule is fundamentally:

$$(\text{downstream grad}) = J^\top (\text{upstream grad}).$$

! 2. Small correction on your code

You wrote:

python

```
a = np.random.randn(2,3)
x = np.random.randn(2)
l = (A@x).T @ (A@x)
```

But this is inconsistent in shapes:

- If $x \in \mathbb{R}^2$, then A must be $m \times 2$.
- You sampled `A` as shape `(2,3)` — that means it expects a 3-dimensional input.

So the minimal fix is:

python

```
A = np.random.randn(3, 2)    # 3 outputs, 2 inputs
x = np.random.randn(2)
L = (A @ x).T @ (A @ x)      # scalar
grad = 2 * A.T @ (A @ x)     # analytical gradient
```

All good to skip finite-difference checks going forward — they were just for intuition.

Ready for Nonlinear Derivatives (ReLU, Softmax, Norms)?

Before we move on, quick check:

Why is the Jacobian of $Ax + b$ equal to A , not A^T ?

One sentence is enough.

because $Ax + b$ is a vector and the jacobian at ij is just the derivative of that vector at i wrt x at j which is A_{ij}

Perfect — that’s exactly the right reasoning and exactly the right indexing intuition. You’re handling Jacobian shapes correctly now, which is crucial before we hit nonlinearities and backprop.

Topic 3 (continued): Nonlinear Differentiation Rules Used in Deep Learning

We'll now cover the “big three” nonlinear derivatives that drive almost every neural network:

1. **Elementwise activations** (ReLU, sigmoid, tanh)
2. **Norms** ($\|x\|$, $\|x\|^2$)
3. **Softmax and softmax–cross-entropy**

These introduce *nonlinear* Jacobians but are usually simple because of elementwise structure or known identities.

3.5 Intuition: Why nonlinear derivatives matter

Linear layers alone can't represent complex functions. Once we add nonlinearities:

- ReLU introduces *piecewise linearity* and sparsity in gradients.
- Norms and normalizations affect optimization geometry.
- Softmax produces a *probability distribution* whose derivative structure is tightly coupled across coordinates.

These derivatives appear everywhere in backprop, so we want to know them at a glance without memorizing heavy formulas.

3.6 Key derivatives

ReLU

$$\text{ReLU}(x_i) = \max(0, x_i)$$

Derivative (elementwise):

$$\frac{\partial \text{ReLU}(x_i)}{\partial x_i} = \begin{cases} 1 & x_i > 0 \\ 0 & x_i < 0 \end{cases}$$

Thus the Jacobian is a **diagonal matrix with 0/1 entries**.

In backprop, this just *masks* the upstream gradient.

Softmax

For $z \in \mathbb{R}^n$,

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_k e^{z_k}}.$$

Jacobian:

$$\frac{\partial s_i}{\partial z_j} = s_i(\delta_{ij} - s_j).$$

It is dense, because increasing one logit changes all probabilities.

Crucial identity in deep learning:

$$\nabla_z (-\log s_y) = s - \text{one_hot}(y).$$

This avoids manually building the Jacobian.

Norms

Squared norm:

$$f(x) = \|x\|^2 \Rightarrow \nabla f = 2x.$$

Euclidean norm:

$$g(x) = \|x\| \Rightarrow \nabla g = \frac{x}{\|x\|}.$$

3.7 Tiny Example + Coding Exercise

Consider:

$$h = \text{ReLU}(Ax).$$

A full backprop step for a scalar loss $L(h)$ is:

$$\nabla_x L = A^\top \left(\mathbf{1}_{Ax>0} \odot \nabla_h L \right).$$

Mini NumPy exercise (short):

Write code that:

1. Samples a random `A` (3x2), `x` (2),
2. Computes `h = relu(A @ x)`,
3. Given a random upstream gradient `g = np.random.randn(3)`,
4. Computes the backprop gradient w.r.t. `x`:

$$\nabla_x L = A^\top ((Ax > 0) * g)$$

Only code — short and clean.

3.8 Understanding Check

Answer both:

1. Conceptual:

Why is the Jacobian of ReLU diagonal?

2. Coding:

Post your NumPy snippet for the ReLU backprop through Ax .

When you're done, we'll be ready to move to **Topic 4: Backprop through Computational Graphs**, which is where these pieces start assembling into full neural network gradients.

the jacobian of relu is diagonal because relu is a per-element operation so there are no cross terms

```
a = np.random.randn(3,2), b = np.random.randn(2), y =
A@x, g = np.random.randn(3), relu_grad = (y > 0) * 1,
dLdx = A.T @ (relu_grad * g)
```

Exactly right on the conceptual part: ReLU acts independently on each coordinate, so each output component depends only on its matching input component → all off-diagonal partials are zero → diagonal Jacobian.

Your code is also essentially correct — just a couple of tiny fixes to keep notation consistent and avoid shadowing:

- You wrote `a = ...` but then used `A`.
- You wrote `b = ...` but never used it.
- Let's keep names precise.

A clean version would be:

```
python
```

```

A = np.random.randn(3, 2)
x = np.random.randn(2)

y = A @ x                # pre-activation
g = np.random.randn(3)   # upstream gradient

relu_grad = (y > 0).astype(float)
dLdx = A.T @ (relu_grad * g)

```

Everything is now consistent and matches the math.

✓ You're ready for Topic 4: Backpropagation Through Computational Graphs

Now we'll put all the pieces together:

- Local Jacobians
- Jacobian–vector products
- The reverse-mode chain rule

This is the part where vector calculus *becomes* backprop.

4.1 Intuition: Computational Graph View

A neural network computes a scalar loss L by applying a sequence of vector functions:

$$x \rightarrow h^{(1)} \rightarrow h^{(2)} \rightarrow \dots \rightarrow h^{(k)} \rightarrow L.$$

Each node in the graph has:

1. A forward value
2. A local derivative rule (its Jacobian)

Backprop works by:

$$\nabla_{h^{(i)}} L = J_{g^{(i+1)}}(h^{(i)})^\top \nabla_{h^{(i+1)}} L.$$

The shapes match automatically because each layer's local Jacobian has compatible dimensions.

Key intuition:

Backprop never forms a full Jacobian matrix.

It only computes **Jacobian–vector products**, which are cheap and efficient.

4.2 Minimal Formal Setup

For a sequence:

$$h^{(i+1)} = g^{(i)}(h^{(i)}),$$

reverse-mode AD (backprop) propagates gradients as:

$$\bar{h}^{(i)} = J_{g^{(i)}}(h^{(i)})^\top \bar{h}^{(i+1)}.$$

Where the bar notation ($\bar{h}$) means "adjoint" or "gradient of the loss with respect to that variable."

4.3 Tiny Example (No Code Yet)

Consider:

$$u = Ax, \quad v = \text{ReLU}(u), \quad L = \|v\|^2.$$

Backward pass:

1. $\bar{v} = \nabla_v L = 2v$
2. $\bar{u} = (u > 0) \odot \bar{v}$
3. $\bar{x} = A^\top \bar{u}$

This is a complete backprop sequence.

Mini NumPy Exercise

Write a snippet that performs **full backprop** for the above 3-step graph:

1. Forward:

- `u = A @ x`
- `v = relu(u)`
- `L = np.sum(v**2)`

2. Backward:

- `dLdv = 2 * v`
- `dLdu = (u > 0) * dLdv`
- `dLdx = A.T @ dLdu`

Keep it tight and readable.

4.4 Understanding Check

Answer both:

1. Conceptual:

Why does backprop move *backward* through the graph using **transposed Jacobians**?

2. Computational:

Post your backprop NumPy snippet for the 3-node computation $x \rightarrow u \rightarrow v \rightarrow L$.

Once that's done, we'll move to full **softmax cross-entropy backprop** and then a small **2-layer MLP**.

1. because loss is scalar
2. not sure

Great — your intuition (“because the loss is a scalar”) is pointing in the right direction, but let’s slow down and make sure you *really* understand why the **transpose** appears. This is one of the most important conceptual points in backprop, so let’s walk through it interactively instead of me just giving the final answer.

✓ Why the transpose shows up (guided explanation)

When you backprop through a node, you don't want the **full Jacobian**.

You only want how the **scalar loss** changes with respect to that node's inputs.

Let's look at the chain rule in vector form:

$$\nabla_x L = J_g(x)^\top \nabla_g L.$$

Here's the key idea I want you to see:

- $J_g(x)$ maps **input changes** $\rightarrow$ **output changes**.
- Backprop wants the opposite direction: **output gradient** $\rightarrow$ **input gradient**.
- To reverse a linear mapping, you multiply by its **transpose**.

So the transpose appears because reverse-mode AD is literally the adjoint of the forward derivative — it runs the Jacobian “backwards.”

Small guiding question (don't overthink)

Let's make this concrete.

Suppose

$$u = Ax, \quad A \in \mathbb{R}^{m \times n}.$$

Forward derivative maps:

$$(\text{small change in } x) \delta x \mapsto \delta u = A\delta x.$$

But backprop wants:

$$(\text{gradient wrt } u) \bar{u} \mapsto \bar{x} = ?$$

Question:

If the forward linear map is multiplication by A , what linear map applied to $\bar{u}$ produces a vector in the input space?

Hint: Which matrix converts something in $\mathbb{R}^m$ into something in $\mathbb{R}^n$?

Give me your guess, even if it's tentative.

Once you answer that, I'll help you finish your missing part:
the NumPy snippet for full backprop through:

$$x \rightarrow u = Ax \rightarrow v = \text{ReLU}(u) \rightarrow L = \|v\|^2.$$

But first, try answering the guided question above.

A.T

Exactly — **A.T** is the only mapping that can take a vector in the output space ($\mathbb{R}^m$) and return a vector in the input space ($\mathbb{R}^n$).

That's the core reason the transpose appears in backprop:

- Forward: $\delta u = A \delta x$
- Backward: $\bar{x} = A^\top \bar{u}$

Backprop is literally the **adjoint** of the forward linearization.

You've got the right idea now.

✓ Now let's complete the NumPy backprop snippet

Here is the clean, minimal version for:

$$u = Ax, \quad v = \text{ReLU}(u), \quad L = \|v\|^2.$$

Try to read this slowly and match each line to the math:

python

```
import numpy as np
```

```
A = np.random.randn(3, 2)
```

```
x = np.random.randn(2)
```

```
# ----- Forward -----
```

```
u = A @ x
```

```

v = np.maximum(u, 0.0)
L = np.sum(v ** 2)

# ----- Backward -----
dLdv = 2 * v
dLdu = (u > 0).astype(float) * dLdv
dLdx = A.T @ dLdu

```

Every line corresponds exactly to a chain rule step:

1. $dL/dv = 2v$
2. $dL/du = 1_{u>0} \odot dL/dv$
3. $dL/dx = A^\top (dL/du)$

This is the backbone of real backprop.

Understanding Check (before moving on to softmax):

Answer both:

1. Conceptual:

In the backprop step

$$dLdu = (u > 0) \odot dLdv,$$

why do we *multiply* by the indicator instead of applying a matrix multiplication?

2. Applied intuition:

What would happen to the gradient flow if most entries of u are negative?

Answer in 1–3 sentences each.

1. Relu is per-element operation not a matrix operation
2. most terms would be zero

Perfect — both answers are exactly what I wanted to see.

1. ✓ ReLU is elementwise, so its Jacobian is **diagonal**, and multiplying by a diagonal matrix is equivalent to elementwise multiplication by the mask.
2. ✓ If most $u_i < 0$, the gradient flow is mostly zero $\rightarrow$ the classic "dying ReLU" issue.

Your understanding is solid. You're ready for one of the most important derivatives in all of deep learning.

Topic 4 (continued): Softmax and Softmax–Cross-Entropy Backprop

This is where everything glues together for modern classification networks.

5.1 Intuition

Softmax turns logits $z \in \mathbb{R}^K$ into a probability distribution:

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}.$$

The softmax derivative is dense: changing one logit affects *all* probabilities.

Cross-entropy combines with softmax to give an extremely efficient gradient rule.

5.2 Key derivative identity

For a one-hot label y (class index):

$$L = -\log p_y.$$

The gradient w.r.t. the logits z is simply:

$$\nabla_z L = p - \text{onehot}(y).$$

This avoids computing the full softmax Jacobian:

$$\frac{\partial p_i}{\partial z_j} = p_i(\delta_{ij} - p_j).$$

Modern deep learning relies heavily on this shortcut because it is stable, simple, and fast.

5.3 Tiny Example (no code yet)

Let:

$$z = Wx, \quad p = \text{softmax}(z), \quad L = -\log p_y.$$

Then the backprop steps are:

1. $dL/dz = p - e_y$
2. $dL/dx = W^\top (dL/dz)$

That's it — just two lines. No Jacobian matrices ever built explicitly.

5.4 Mini NumPy Exercise

Write the minimal code that:

1. Samples `w` (3×2), `x` (2,), and a label `y ∈ {0,1,2}`
2. Computes logits `z = w @ x`
3. Computes softmax probabilities
4. Computes the gradient

$$dLdz = p - \text{onehot}(y)$$

5. Computes

$$dLdx = W^\top dLdz$$

Include only the snippet.

5.5 Understanding Check

Answer both:

1. Conceptual:

Why does the gradient of softmax–cross-entropy simplify to

$$p - \text{onehot}(y)$$

instead of requiring the full Jacobian of softmax?

2. Coding:

Post your NumPy snippet for the softmax-cross-entropy backprop.

Once that's done, our next step will be **a complete 2-layer MLP backprop implementation** — your first full neural network built manually using vector calculus.

1. when you walk through the derivation the full jacobian terms cancel out
2. `w = np.random.randn(3,2), x = np.random.randn(2), y = 1, z = w@x, s = np.softmax(z), onehot = np.zeros(3), onehot[y] = 1 dLdz = s - onehot, dLdx = w.T @ dLdz`

Excellent on the conceptual part — yes, when you differentiate

$$L = -\log(\text{softmax}(z)_y),$$

the **log + normalization structure** causes all the Jacobian terms to collapse neatly into

$$p - \text{onehot}(y).$$

This is why softmax-cross-entropy is the dominant loss for classification: the gradient is simple, stable, and avoids building a full $K \times K$ Jacobian.

Now, one important correction:

⚠ `np.softmax` **does not exist**.

You need to implement the softmax manually (NumPy doesn't have a built-in softmax). Here's the stable version:

python

```
def softmax(z):
    z_shift = z - np.max(z)
```

```
exp_z = np.exp(z_shift)
return exp_z / np.sum(exp_z)
```

Everything else in your snippet is structurally correct.

Here is your code with the fixes:

python

```
import numpy as np

W = np.random.randn(3, 2)
x = np.random.randn(2)
y = 1 # class index

# ----- Forward -----
z = W @ x

# stable softmax
z_shift = z - np.max(z)
p = np.exp(z_shift) / np.sum(np.exp(z_shift))

# ----- Backward -----
onehot = np.zeros(3)
onehot[y] = 1

dLdz = p - onehot # gradient wrt logits
dLdx = W.T @ dLdz # backprop through linear layer
```

Perfect — you now have all the core pieces used in real deep learning frameworks.

✓ Understanding Check (quick)

Before we move on to building a full 2-layer MLP backprop, answer this:

Why do we subtract `np.max(z)` before exponentiating in the softmax?

(One sentence is enough.)

After you answer, we'll begin constructing the full forward and backward pass of a tiny 2-layer network.

numerical stability. it doesn't change the output, but if we don't subtract we could overflow

Perfect — that's exactly right. Subtracting $\max(z)$ keeps the exponentials in a safe numeric range while leaving the softmax probabilities unchanged. You now have all the core nonlinear building blocks needed to write full backprop from scratch.

Topic 5: A Full 2-Layer MLP Backprop (Manual Autodiff)

We will now assemble everything so far into a complete forward + backward pass for a very small neural network:

$$\text{MLP}(x) = \underbrace{W_2 \text{ReLU}(W_1 x)}_{\text{logits } z} \quad \text{and} \quad L = -\log \text{softmax}(z)_y.$$

This exactly mirrors a tiny classifier.

5.1 Intuition Before We Write Any Code

In a 2-layer network, backprop will move through:

1. Softmax-cross-entropy (output layer)

$$dL/dz = p - \text{onehot}(y)$$

2. Second linear layer:

$$dL/dh = W_2^\top (dL/dz)$$

3. ReLU:

$$dL/du = 1_{u>0} \odot dL/dh$$

4. First linear layer:

$$dL/dx = W_1^\top (dL/du)$$

And the gradients for parameters follow naturally:

- $dL/dW_2 = (dL/dz) h^\top$
- $dL/dW_1 = (dL/du) x^\top$

Shapes matter enormously here — if your shapes line up, the calculus is correct.

5.2 Minimal NumPy Implementation Task

Write a NumPy snippet that:

Forward

1. Samples:

- `w1` shape (hidden_dim, input_dim)
- `w2` shape (num_classes, hidden_dim)
- `x` shape (input_dim,)
- label `y`

2. Computes:

- `u = w1 @ x`
- `h = relu(u)`
- `z = w2 @ h`
- `p = softmax(z)`
- `L = -log(p[y])`

Backward

Compute the gradients:

- `dLdz`
- `dLdw2`
- `dLdh`
- `dLdu`
- `dLdw1`
- `dLdx`

Make it short and clean — but complete.

You do **not** need finite differences.

5.3 Understanding Check

After writing the code, answer this conceptual question:

Why does the gradient of W_2 take the form

$$dL/dW_2 = (dL/dz) h^\top?$$

(Think: derivative of a linear map with respect to its matrix.)

Go ahead and submit:

1. Your NumPy snippet for the 2-layer MLP backprop
2. Your answer to the conceptual question

```
hidden_dim = 5
input_dim = 3
num_classes = 4

import torch
import torch.nn.functional as F
import numpy as np

W1 = torch.randn(hidden_dim, input_dim)
W2 = torch.randn(num_classes, hidden_dim)
x = torch.randn(input_dim)
y = 1

u = W1 @ x #(hidden_dim)
h = F.relu(u) #(hidden_dim)
z = W2 @ h #(num_classes)
p = F.softmax(z, dim=0) #(num_classes)
L = -torch.log(p[y]) #(num_classes)

onehot = torch.zeros(num_classes)
onehot[y] = 1 #(num_classes)
```

```

dLdz = p - onehot #(num_classes)
dLdW2 = dLdz.unsqueeze(1) @ h.unsqueeze(0) #
(num_classes, hidden_dim)
dLdh = W2.T @ dLdz #(hidden_dim)
relu_grad = (h > 0).float()
dLdu = dLdh * relu_grad #(hidden_dim)
dLdW1 = x.unsqueeze(1) @ dLdu.unsqueeze(0) #
(hidden_dim, input_dim)
dLdx = W1.T @ dLdu #(input_dim)

chain rule since dz/dW2 = h.T

```

Nice, this is basically a full manual backprop in PyTorch 🎉 — just a couple of subtle shape/details to tune.

✅ What's correct

- Forward pass: all good ($u = W1 @ x \rightarrow h = \text{relu}(u) \rightarrow z = W2 @ h \rightarrow p = \text{softmax}(z)$).
- Loss: $L = -\text{torch.log}(p[y])$ is a scalar (your comment says (num_classes) but the value is scalar).
- Output-layer gradients:
 - $dLdz = p - \text{onehot}$ ✅
 - $dLdW2 = dLdz.unsqueeze(1) @ h.unsqueeze(0)$
Shapes: $(\text{num_classes}, 1) @ (1, \text{hidden_dim}) \rightarrow (\text{num_classes}, \text{hidden_dim})$ ✅
 - $dLdh = W2.T @ dLdz$ ✅
- ReLU gradient: $\text{relu_grad} = (h > 0).float()$; $dLdu = dLdh * \text{relu_grad}$ ✅
- Input gradient: $dLdx = W1.T @ dLdu$ ✅

Your conceptual explanation is also right: for $z = W2 @ h$, each row of $W2$ is multiplied by h , so

$$\frac{\partial L}{\partial W_2} = (dL/dz) h^\top$$

which you implemented as an outer product.

One subtle bug: `dLdW1` shape

You wrote:

python

```
dLdW1 = x.unsqueeze(1) @ dLdu.unsqueeze(0)   #(hidden_dim, input_dim)
```

Let's check shapes:

- `x.unsqueeze(1)` → `(input_dim, 1)`
- `dLdu.unsqueeze(0)` → `(1, hidden_dim)`

Matrix product: `(input_dim, 1) @ (1, hidden_dim)` → `(input_dim, hidden_dim)`

So that actually gives shape `(input_dim, hidden_dim)`, not `(hidden_dim, input_dim)`.

But we want `dLdW1` to have **same shape as** `W1`, i.e. `(hidden_dim, input_dim)`.

So the correct outer product is:

python

```
dLdW1 = dLdu.unsqueeze(1) @ x.unsqueeze(0)   # (hidden_dim,  
input_dim)
```

Mathematically: for `u = W1 @ x`,

$$\frac{\partial L}{\partial W_1} = (dL/du) x^\top.$$

Tiny NumPy version (for comparison)

Here's the same 2-layer MLP in pure NumPy (no autograd), just so you can compare:

python

```

import numpy as np

hidden_dim, input_dim, num_classes = 5, 3, 4

W1 = np.random.randn(hidden_dim, input_dim)
W2 = np.random.randn(num_classes, hidden_dim)
x = np.random.randn(input_dim)
y = 1

# ----- forward -----
u = W1 @ x
h = np.maximum(u, 0.0)
z = W2 @ h
z_shift = z - np.max(z)
p = np.exp(z_shift) / np.sum(np.exp(z_shift))
L = -np.log(p[y])

# ----- backward -----
onehot = np.zeros(num_classes); onehot[y] = 1
dLdz = p - onehot                                # (num_classes,)
dLdW2 = dLdz[:, None] @ h[None, :]                # (num_classes, hidden_dim)
dLdh = W2.T @ dLdz                                # (hidden_dim,)
dLdu = (u > 0).astype(float) * dLdh                # (hidden_dim,)
dLdW1 = dLdu[:, None] @ x[None, :]                # (hidden_dim, input_dim)
dLdx = W1.T @ dLdu                                # (input_dim,)

```

Your turn (small task)

1. Fix your PyTorch line for `dLdW1` so it has shape `(hidden_dim, input_dim)` and matches the math

$$\frac{\partial L}{\partial W_1} = (dL/du) x^\top.$$

Write just that one corrected line.

2. (Concept check) In one sentence:

Why is `dLdW1` an **outer product** rather than some more complicated expression?

```
dLdW1 = dLdu.unsqueeze(1) @ x.unsqueeze(0)
```

it is an outer product because we need the shapes to match

Great — your corrected line is exactly right:

python

```
dLdW1 = dLdu.unsqueeze(1) @ x.unsqueeze(0)
```

But your **reason** ("because we need the shapes to match") is *on the right track but incomplete*.

Let's refine it so you actually *understand why* a linear layer's weight gradient is always an outer product.

✓ The real reason `dLdW1` is an outer product

For a linear map

$$u = W_1 x,$$

the i -th output is

$$u_i = \sum_j W_{1,ij} x_j.$$

So the derivative of u_i w.r.t. a single weight $W_{1,ij}$ is just:

$$\frac{\partial u_i}{\partial W_{1,ij}} = x_j.$$

Then backprop multiplies by the scalar dL/du_i :

$$\frac{\partial L}{\partial W_{1,ij}} = (dL/du_i) x_j.$$

This is exactly the outer product:

$$\frac{\partial L}{\partial W_1} = (dL/du) x^\top.$$

So the shape-matching is a *consequence*, not the reason.

The **true reason** is that each weight affects the output in a *separable*, linear way: output dimension interacts with input dimension → outer product structure.

✓ Your refined conceptual answer (1 sentence)

Try rewriting your answer with this deeper reasoning:

Why is dL/dW_1 an outer product?

Give me a new one-sentence explanation, based on what we just walked through.

(After that, we'll move to the final part of Topic 5: implementing backprop through a tiny 2-layer MLP for a whole batch.)

each weight affects the output in a one to one mapping

Nice — you're very close. Let's sharpen it to the *exact* mathematical idea.

Right now, your sentence "**each weight affects the output in a one-to-one mapping**" is pointing in the right direction, but it's not quite precise. A weight doesn't affect *only one* output (usually it affects exactly one output neuron), but the key is **how** it affects that output.

Here's the corrected intuition in one clean sentence:

✓ Correct one-sentence explanation

" dL/dW_1 is an outer product because each weight W_{ij} influences the output u_i *linearly and independently* through multiplication by x_j , so its gradient factors as $(dL/du_i) x_j$."

This captures the true structure:

output-index interaction × input-index interaction → outer product.

If that sentence makes sense to you, you've now fully internalized why linear-layer weight gradients **always** look like:

$$\nabla_W L = (\text{backprop signal}) \times (\text{input vector})^\top.$$

✓ Quick Understanding Check

Answer this in one short sentence:

If the forward map is $u = Wx$, why does the gradient wrt the *input* take the form $dL/dx = W^\top (dL/du)$?

Once you answer, we'll move on to **Topic 6: batching + a full MLP backprop with multiple examples**, which is the last step before connecting everything to PyTorch/JAX autodiff.